

Pour Pal 2.0

Final Project

**Carter Hill
Daniel Nalepa
Adam Slamani**

CMPE 3815
University of Vermont
12/12/2025

Contents

1	Introduction	1
2	System Overview	2
3	Hardware	3
4	Web Interface	4
4.1	Overview	4
4.1.1	Manual Control Panel	4
4.1.2	Custom Cocktail Creation	5
4.1.3	System Status	6
5	Packaging	7
5.1	Packaging Design	7
5.2	Using Onshape	8
5.3	Packaging Design Features	11
6	Code	11
7	Discussion	16
8	Work Distribution	16
9	Conclusion	16
10	Appendix	18

Abstract

The Poor Pal 2.0 is a wireless drink dispenser designed to automatically pour beverages. Using an Arduino Nano ESP32, the user will interface through a web server over WiFi to request a specific volume of available or custom cocktails. Six peristaltic pumps controlled through L298N motor drivers will precisely dispense the requested quantities. An LCD screen will provide real-time feedback on system status. The device shall be in a compact package.

1 Introduction

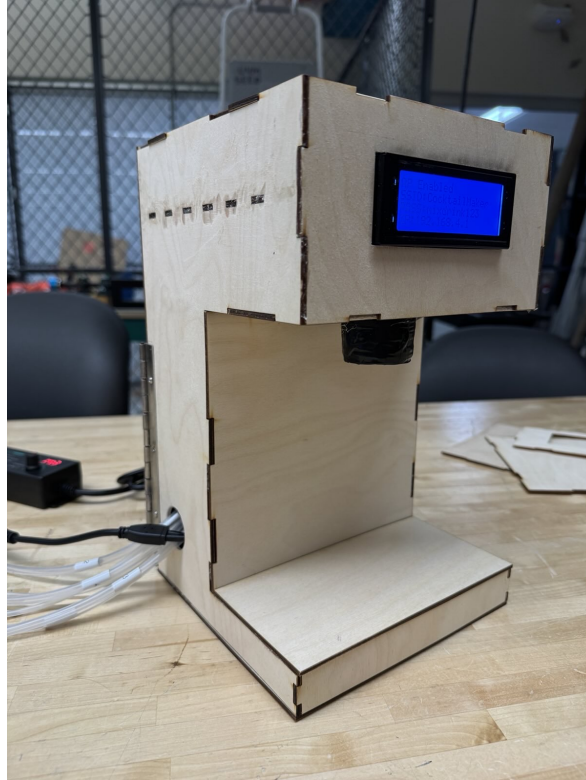


Figure 1: Pour Pal 2.0

The Pour Pal 2.0 project focuses on designing and building a compact, WiFi-controlled drink-dispensing system that can mix custom beverages using a combination of peristaltic pumps, motor drivers, and an ESP32 microcontroller. The goal of this version was to create a cleaner, more reliable, and more user-friendly machine compared to earlier prototypes. To support this, the system uses a web-based interface for selecting drinks and an LCD screen on the front panel that provides live feedback while the machine is running.

A major part of Pour Pal 2.0 was developing a functional and well-packaged enclosure. The entire frame and exterior were laser-cut from wood, allowing the panels to interlock cleanly while still being easy to assemble and modify. Features like the removable roof, hinged back door, and dedicated cutouts for tubing and power were all included to make the machine easier to use and maintain. Overall, this project highlights the process of taking an early concept and refining it into a well-packaged, fully integrated system.

2 System Overview

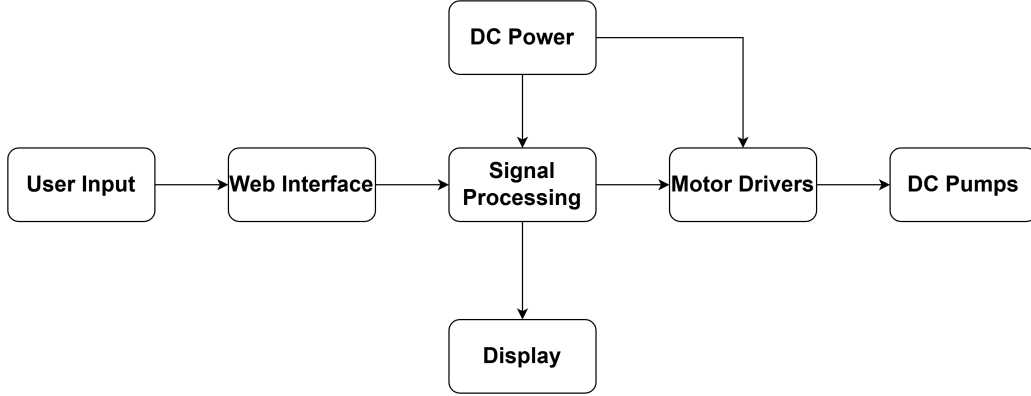


Figure 2: Pour Pal 2.0 Block Diagram

As shown in Figure 2, the Pour Pal 2.0 functions as a WiFi-enabled automated drink dispenser. Users connect to the system through a locally hosted web interface, where they can manually control individual pumps or select from stored custom cocktail recipes. When a user submits a drink request, the Arduino Nano ESP32 processes the input, displays system status on the front-mounted LCD, and activates the correct combination of peristaltic pumps.

Each pump draws liquid from a dedicated external container through an intake line, ensuring that ingredients never mix before dispensing. The requested volumes are converted into pump run-times based on the calibrated flow rates of the 100 mL/min and 220 mL/min pumps. Three L298N motor drivers, each responsible for two pumps, receive digital control signals from the ESP32 to start, stop, and reverse pump directions.

The system is powered by a 14.6 V supply, which provides sufficient overhead for the motor driver voltage drop while still maintaining consistent pump performance. Internally, power is distributed to the ESP32, the LCD, and all motor drivers through a combination of regulated 5 V logic and higher-voltage motor rails.

Together, the web interface, microcontroller, pump assembly, and power system form a fully integrated drink-dispensing device that is compact, easy to operate, and capable of producing repeatable, customizable beverage mixes.

3 Hardware

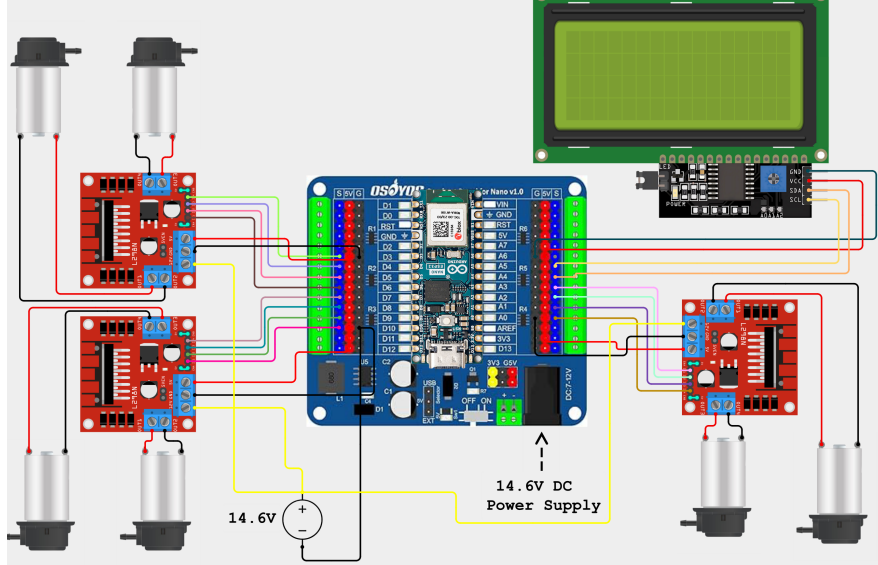


Figure 3: Circuit Diagram

Figure 3 demonstrates the equivalent circuit of the Pour Pal 2.0. The microcontroller of choice is an Arduino Nano ESP32, as this allows for support of WiFi and web servers. This is mounted on an OSOY00 Nano Breakout Board for easy access to pins and a neater package. To power the board, a variable AC/DC adapter is used to allow the user to simply plug the machine into an outlet like any other drink dispenser.

As this project uses six 12V peristaltic pumps, with four pumps having a flow rate of 220 ml/min and the other two with a flow rate of 100 ml/min, three L298N motor driver boards are needed, as each one can drive two motors. The logic of these boards is powered by 5V taken directly from the breakout board, while the external power supply provides voltage to the pumps. To achieve the rated 12V at each pump, it is required to supply 14.6V to the boards to account for the voltage drop across various components. This circuit runs at a few amps when all pumps are running; therefore, even just a little resistance from a wire or circuit element can cause a few volts to be lost. The motor driver contains two separate H-bridges that independently control each pump based on the digital logic. The four input pins on each motor driver board are connected to digital pins on the Arduino and set to logic HIGH or LOW, biasing the H-bridge to spin the pump one way or another. Analog pins zero through three are used as digital pins due to an insufficient number of available digital pins on the Nano. Additionally, both enable pins on the board are directly jumped to 5V, as for this application, the pumps do not need to be turned off and on, and it also helps free more pins on the microcontroller.

Lastly, an LCD is used to display the access point (AP), which is when the Arduino creates its own WiFi network and is ready for users to connect. Additionally, it displays the service self-identifier (SSID), password, and IP address to access the web page. LCDs use something called I2C communication, which is a two-wire serial communication protocol that allows the microcontroller to communicate with the screen. The LCD is powered by 5V from the breakout board and connects SDA to A4 and SCL to A5, as these are the serial data and serial clock line of the microcontroller.

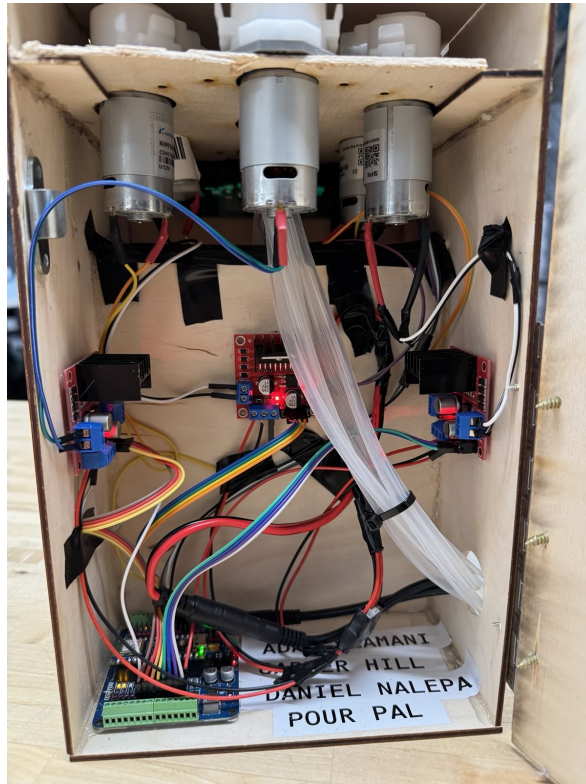


Figure 4: Constructed Circuit

The image above demonstrates the constructed circuit housed inside the packaging.

4 Web Interface

4.1 Overview

This section goes into detail regarding the web interface design and its implementation. The Pour Pal is controlled using a custom web interface that is hosted on the Arduino Nano ESP32 microcontroller. Using a WiFi AP, users can connect to the system from any device as long as they can connect to its WiFi using the SSID and password displayed on the LCD. This is beneficial since no external network is needed, allowing the Pour Pal to be portable. Once a user is connected to the WiFi, they can access it through a web browser like Google Chrome by searching the ESP32's local IP address: <http://192.168.4.1/>. This system uses the ESP32 WebServer library to host a HTTP server on port 80. This gives the Pour Pal the ability to handle client requests and support GET and POST HTTP methods for easy interaction with the web server. GET is primarily used for getting data from a server, whether it be data in a URL or cacheable requests for fast repeat scenarios, like in the timer. On the other hand, the POST is primarily used for giving data to the web server to modify a variable like the state of the peristaltic pumps and up-time timer. These are not cacheable and are not visible in the URL. Overall, the web server is able to route URL endpoints to its many handler functions for completing tasks such as dispensing drinks, saving drink recipes, and updating system status.

4.1.1 Manual Control Panel

The first part of the web interface shows a manual control panel as seen below:

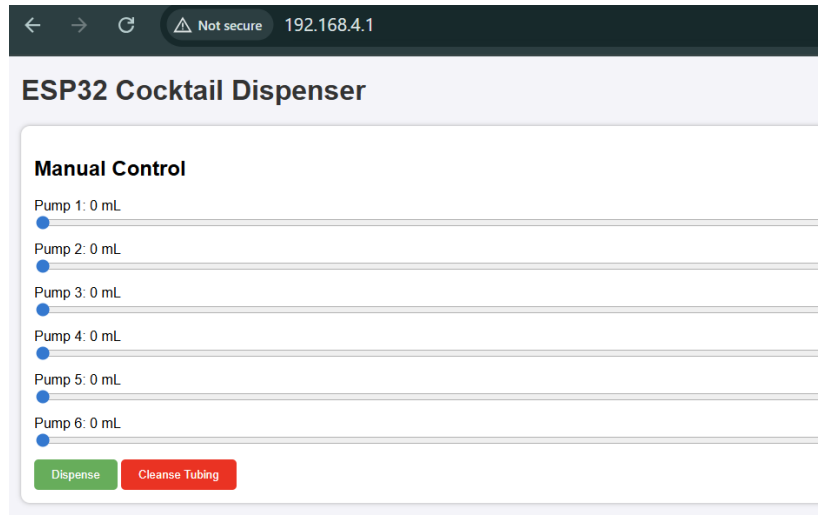


Figure 5: Web Interface: Manual Control Panel

This panel allows the user to directly and precisely control the amount of liquid that can be poured by the peristaltic pumps. Each pump has its own slider, which gives the user input for selecting their desired volume between 0-100 ml. This range was arbitrarily chosen because this machine makes cocktails and will never dispense large amounts of a single ingredient. When the slider is adjusted, the label and level are automatically updated using JavaScript event listeners, which gives users real-time control and confirmation of their choices. There are also two selectable buttons at the bottom of the panel labeled "Dispense" and "Cleanse Tubing". The first button dispenses the selected liquids and their ranges by gathering the slider value for each pump and appending them as URL parameters to a GET request sent by the Microcontroller. Each pump volume is then independently converted into a calibrated dispense time based on the flow rate for each respective pump. This gives the machine the ability to dispense any pump and pump volumes configuration, allowing the user to have a lot of freedom when designing their cocktails. The other button is used to maintain the cleanliness of the silicon tubing and to remove excess liquids from it by reversing the pump's flow. When pressed, a command is sent to the "cleanse" endpoint in the code. This will tell the pumps to reverse the direction of flow for a short interval of time before turning off. This gives the system the ability to flush its lines, preventing gunk buildup and cross-contamination between different liquids if one recipe is used after another.

4.1.2 Custom Cocktail Creation

In this next subsection, the custom cocktails are explained. Below is a picture of the custom cocktail panel, as well as the Stored Cocktail panel and Status panel:

The screenshot displays a web interface with three distinct panels. The top panel, titled 'Create Custom Cocktail', contains a 'Cocktail Name' input field, a 'Pump Liquid Names (1-6)' section with six input fields labeled 'Pump 1 name' through 'Pump 6 name', and a 'Pump Volumes (0-100 mL)' section with six input fields. A green 'Save Cocktail' button is positioned at the bottom right of this panel. The middle panel, titled 'Stored Cocktails', features a scrollable list of cocktail names: 'Daniel', 'Daniel', 'Adam', and 'Adam'. Below the list are two green buttons: 'Dispense Selected' and 'Delete Selected'. The bottom panel, titled 'About / Status', provides system information including 'IP Address: 192.168.4.1', 'Uptime: 00:03:40', 'WiFi: Access Point', and a 'Pumps' section listing six pumps, all of which are currently 'Idle'.

Figure 6: Custom Cocktail, Stored Cocktail, and Status Panels

The custom cocktail panel allows the user to design and store up to ten custom drink recipes. There are three categories for user input, as seen below, that are collected into a form:

1. **Cocktail Name:** A string used to identify the custom drink.
2. **Pump Liquid Names:** Labels for the liquids connected to each of the six pumps.
3. **Ingredient Volumes:** The amount in mL each pump should dispense when the recipe is selected.

Once the user inputs these selections and presses the "Save Cocktail" button, the values are placed into a JavaScript object, serialized into JSON, and transmitted to the Microcontroller with a POST request to the "saveCocktail" endpoint. POST is used here since the data is being modified, changing the state of the current system without changing the URL. When the JSON data is received, it is parsed, and the recipe is stored in the onboard non-volatile flash memory utilizing Arduino's Preferences library. This allows recipes to be stored even when the Pour Pal is powered off. This provides an added level of convenience for the user.

Moving on, the next panel of the web interface deals with managing saved cocktails. Here, the user can select a stored cocktail and either dispense it or delete the recipe. This is done by a periodic GET request to the "getCocktails" endpoint that lists the array of all the stored recipes on the web page. Below are the two operations and their endpoints:

- **Dispense Stored Cocktail:** GET request to "dispenseCocktail"
- **Delete Cocktail:** POST request to "deleteCocktail" to remove the recipe from flash memory.

This system improves the user experience when preparing the same drink repeatedly.

4.1.3 System Status

The last panel on the web page is for the system's status. This section gives the user or system designer the ability to monitor in real-time the Pour Pals state regarding three parameters: WiFi Mode and IP Address, System Uptime, and Pump Status Indicators. The first parameter is for

internet-related status updates and confirms to the user that the Pour Pal is connected and gives the IP address of the web server. The second status gives the total run time since the web server has booted. The third shows each pump's status indicators, which are either labeled as running or idle. All of this information is refreshed every second using a JavaScript timer and is asynchronous, so that the user never has to refresh the page manually for a better user experience.

5 Packaging

5.1 Packaging Design



Figure 7: Front View of the Pour Pal 2.0

The enclosure for the Pour Pal 2.0 was designed in Onshape and built using 3 mm laser-cut wood. The primary objectives for the packaging were to design a layout that is intuitive for the user, easy to assemble, and straightforward to troubleshoot during testing. Its overall dimensions are 18cm x 18cm x 30cm, with a drink shelf that is 18cm x 9cm x 17.5cm and can fit a standard pint glass. The final design resembles a compact drink dispenser, which gives the system a familiar look and makes the interface easy to understand without instructions.

Inside the enclosure, all electronics are mounted in a way that keeps wiring organized and provides clear access to the pumps, motor drivers, and microcontroller. Positioning the pumps higher than the liquid sources prevents siphoning issues and makes pump replacement or troubleshooting much easier. The LCD is placed at the front of the unit so that the SSID, password, and system status are always visible to the user (*Figure 7*).

The intake tubing enters on the lower left side of the housing. This allows the liquid containers to sit on a lower shelf or directly beside the Pour Pal 2.0, keeping the workspace uncluttered while making

it easy to swap out bottles. Overall, the packaging design supports both usability and functionality by protecting the electronics, keeping the system compact, and making daily operation straightforward.

5.2 Using Onshape

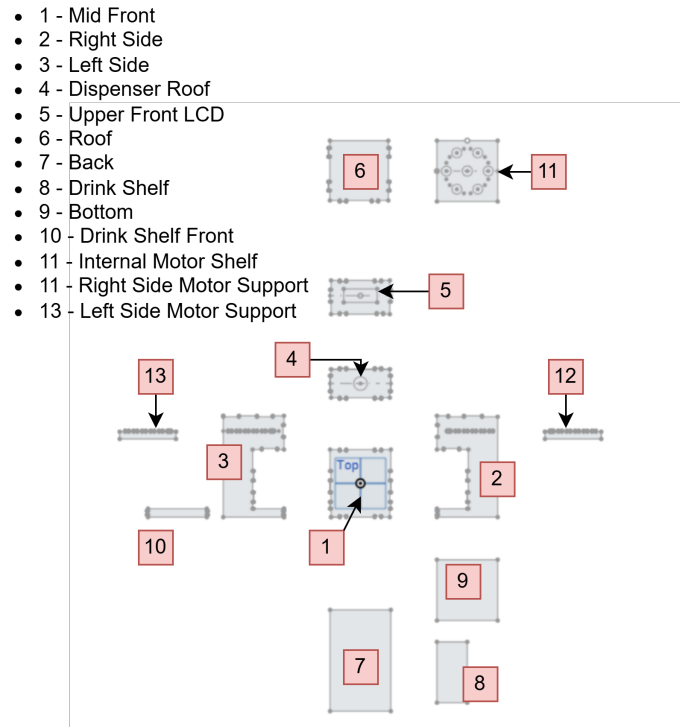


Figure 8: CAD designs of the packaging

As shown in Figure 8, each part was created as a 2D sketch to be laser-cut. Teeth were added along the edges of the structural panels to improve alignment, stability, and ease of assembly. An example of these interlocking edges is shown in Figure 9, where the central front piece and the right side panel align using protruding 3 mm teeth and matching notches of the same depth. These features add enough surface area for effective gluing while remaining thick enough not to snap during assembly.

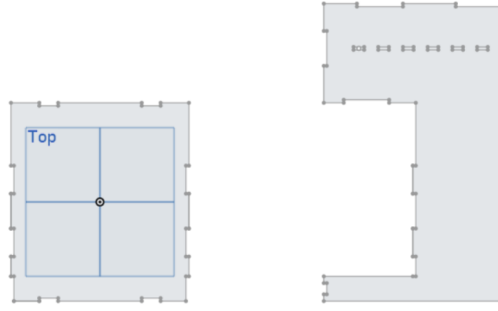


Figure 9: An Example of Wood Panel Edge Alignment in Onshape

To ensure both edges were the same size, a series of geometric constraints were applied to the sketch. The first shape created was a central rectangle at the origin of the workspace. The side panel was then created as a corner-point rectangle, offset slightly from the mid-front panel so that the bottom edges aligned. Lines defining the side panel's outline were drawn using the line tool (shortcut ****l****) and trimmed using the trim tool (shortcut ****m****), as shown in Figure 10. Grooves were then added to the mid-front panel using the same method, ensuring its overall width remained 18 cm.

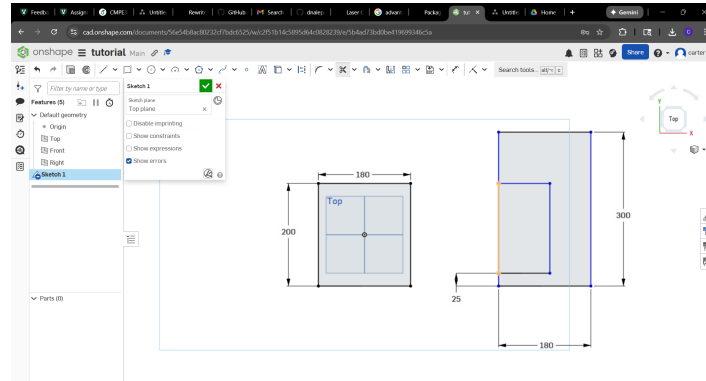


Figure 10: Onshape CAD example with the mid front panel and the right side panel (dimensions are in mm)

The insets were set to equal width using the “Equal” constraint (shortcut ****e****). The completed edge was then mirrored to the opposite side of the mid-front panel. After unnecessary lines were removed, this edge profile was copied and applied to the right panel, as shown in Figure 11, ensuring the teeth align perfectly. Additional insets were added at the bottom corners to account for the lower edge of the side panel.

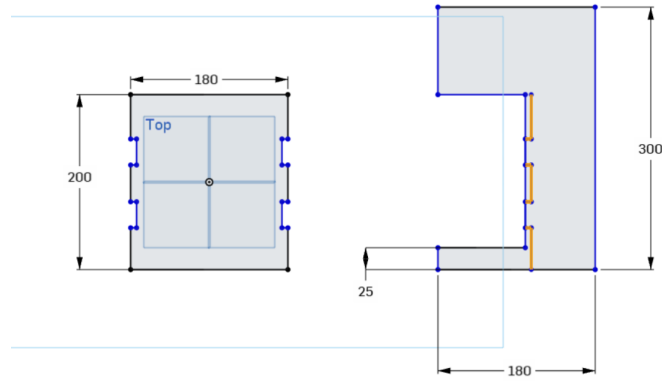


Figure 11: The right edge of the mid front panel pasted to the edge of the right side panel

This process was repeated for each applicable panel (1, 2, 4, 5, 6, and 10). The motor shelf was created using the Circular Pattern tool. A square base with a central hole was sketched first. Based on motor measurements, a 29 mm diameter hole was added, along with two 3.2 mm diameter mounting holes spaced 24.25 mm from its center. The Circular Pattern tool was then used to create six evenly spaced motor cutouts, as shown in *Figure 12*.

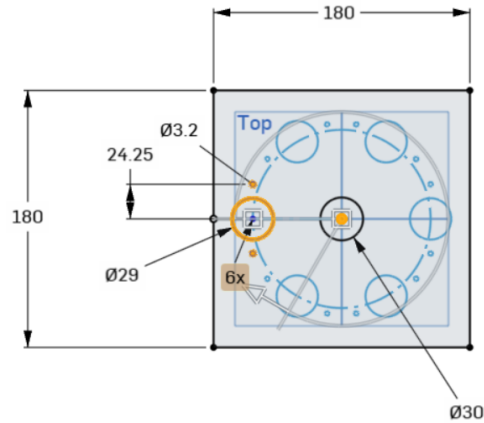


Figure 12: An example of the Circular Pattern tool in Onshape

Using a similar tool called "Linear Pattern," the right and left motor shelves and their corresponding insets were also sketched (parts 12 and 13). Once fully dimensioned, the necessary panels were reflected using the mirror to create their corresponding left-sided parts (3 and 13). The final sketch was then downloaded in a .DWG format and sent to the UVM Fabrication Lab for laser cutting.

5.3 Packaging Design Features

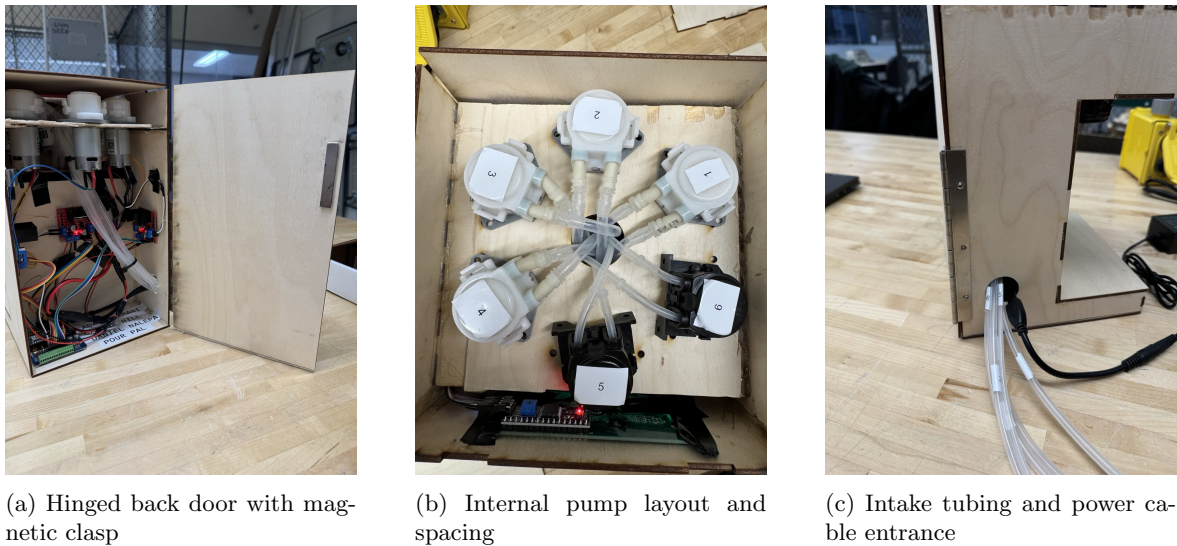


Figure 13: Key design features of the Pour Pal 2.0 housing

The housing includes several design features intended to make the system easy to use and maintain. The roof panel is fully removable, giving quick access to the pumps and wiring during testing or repairs. A hinged back door with a small magnetic clasp keeps the enclosure closed during operation, but can be opened when needed. Inside the housing, the three motor drivers are mounted on separate internal panels and spaced apart to prevent overheating and simplify wiring. A dedicated cutout on the left side provides a clean path for the intake tubing and the power cable, helping to keep the workspace organized and reducing strain on the connections.

6 Code

```
1 // Daniel Nalepa, Carter Hill, Adam Slamani
2 // Pour Pal Cocktail Dispenser
3 // ESP32 Access Point with Flash Storage
4
5 //Import libraries
6 #include <WiFi.h> // Create WI-FI Functionality
7 #include <WebServer.h> // Create Web Server
8 #include <ArduinoJson.h> // JSON Formatting Data
9 #include <LiquidCrystal_I2C.h> // I2C Communication
10 #include <Preferences.h> // Flash Storage
```

Initially, all necessary libraries are included. `Wifi.h` enables network connection for the Arduino, `WebServer.h` allows for the creation of an embedded web server, `ArduinoJson.h` is used for storing and retrieving data in JSON structure to save user cocktails, `LiquidCrystal_I2C.h` allows for I2C communication for the LCD, and `Preferences.h` allows for the use of flash memory.

```
1 // Generate Web Server
2 WebServer server(80);
```

```

3
4 // LCD Address and Screen Size
5 LiquidCrystal_I2C lcd(0x27, 20, 4);
6
7 // Define Motor Driver IN pins (Each pump uses 2 input pins)
8 const int inPins[12] = {3,4,5,6,7,8,9,10,A0,A1,A2,A3};
9
10 // Flow Rate Calibration Factors mL/sec
11 const float pumpFlowRate[6] = {3.3, 3.3, 3.3, 3.3, 1.6, 1.6};
12
13 // Define Pump State
14 int pumpVolumes[6] = {0,0,0,0,0,0};
15 bool pumpRunning[6] = {false,false,false,false,false,false};
16 unsigned long pumpEndTime[6] = {0,0,0,0,0,0};

```

Next, the web server is generated, and the LCD address is defined. Following this, a few arrays are created that store the motor driver input pins, flow rate correction factors, and the current state of the pumps.

```

1 // Store User Cocktails
2 #define MAX_COCKTAILS 10
3 Cocktail storedCocktails[MAX_COCKTAILS];
4 int cocktailCount = 0;
5
6 //Define Variable to Track Uptime
7 unsigned long startTime;
8
9 // Creates Object for Flash Memory
10 Preferences prefs;

```

As this project has the functionality to store custom drinks, the structure of what is stored needs to be defined. Here, struct Cocktails stores the name, the amount of liquid volume from each pump, and the names of the liquids. These will then be stored in the array storedCocktails, where up to 10 unique cocktails can be stored. This data is read from and written to the flash memory.

```

1 // Calculates Uptime and Returns String to Display to Webpage
2 String getUptime() {
3     unsigned long ms = millis() - startTime;
4     unsigned long s = ms / 1000;
5     unsigned long m = s / 60;
6     unsigned long h = m / 60;
7     s %= 60;
8     m %= 60;
9     char buf[32];
10    sprintf(buf, "%02lu:%02lu:%02lu", h, m, s);
11    return String(buf);
12 }

```

The function here uses millis to calculate and return the up-time of the system in the format HH:MM:SS.

```

1 // Indexing Structure
2 // 2*idx first pint, 2*idx +1 second pin

```

```

3 // indx 0 --> Pump1 --> IN Pins 3/4
4 // indx 1 --> Pump2 --> IN Pins 5/6
5 // indx 2 --> Pump3 --> IN Pins 7/8
6 // indx 3 --> Pump4 --> IN Pins 9/10
7 // indx 4 --> Pump5 --> IN Pins A0/A1
8 // indx 5 --> Pump6 --> IN Pins A2/A3
9
10 // Stops Pumps
11 void hardwareStopPump(int idx) {
12     digitalWrite(inPins[2*idx], LOW);
13     digitalWrite(inPins[2*idx + 1], LOW);
14 }
15
16 // Run Pumps to Dispense
17 void hardwareStartPump(int idx) {
18     digitalWrite(inPins[2*idx], LOW);
19     digitalWrite(inPins[2*idx + 1], HIGH);
20 }
21
22 // Run Pumps to Retract
23 void hardwareReversePump(int idx) {
24     digitalWrite(inPins[2*idx], HIGH);
25     digitalWrite(inPins[2*idx + 1], LOW);
26 }
27
28 // Runs Pump for Defined Amount of Fluid
29 void startPump(int idx, int volume_mL) {
30     if(idx<0||idx>=6||volume_mL<=0) return; // Checks for Correct Indexing
31     pumpVolumes[idx] = volume_mL; // Store Requested Volume for said Pump
32     pumpRunning[idx] = true; // Set state of Pump running to true
33     unsigned long duration = (unsigned long)((volume_mL / pumpFlowRate[idx]
34         )*1000.0); // Calculate Time to Run Pump
35     pumpEndTime[idx] = millis() + duration; // Future Time to Stop Pump
36     hardwareStartPump(idx); // Run Pump
37 }
38 // Stop Pump and Reset State
39 void stopPump(int idx) {
40     if(idx<0||idx>=6) return; // Checks for Correct Indexing
41     pumpRunning[idx] = false; // Set state of Pump running to false
42     pumpVolumes[idx] = 0; // Reset Requested Volume
43     pumpEndTime[idx] = 0; // Reset Future Time
44     hardwareStopPump(idx); // Stop Pump
45 }

```

The functions here are responsible for controlling the functionality of the pumps. As each motor is controlled by two input pins, an indexing system from 0 through 5 corresponds to pumps 1 through 5. By using the values of $2*idx$ and $2*idx+1$ to access the `inPins` array, the correct pins are used to bias the pump. `hardwareStopPump` stops the pump immediately by pulling both input pins LOW, `hardwareStartPump` runs the pump forward by pulling one pin HIGH and the other LOW, `hardwareReversePump` runs the pump in reverse by doing the opposite of `hardwareStartPump`. Additionally, the function `startPump` passes through the index value and the volume requested. It then calculates

the duration for how long the pump should run and calls `hardwareStartPump`. On the other hand, `stopPump` resets the state of the pumps and stops them from running.

Due to the team's limited prior experience with microcontroller flash memory and HTML web server programming, and the narrow timeline, the team utilized the assistance from AI to help develop these features. Using our specific hardware, pre-existing code, and guidance after multiple iterations, the team acquired code that stores the user-defined cocktails on the Arduino's flash memory and operates through an interactive web interface. This section of the code is located in the Appendix below.

HTML is the standard language for creating webpages. Using this, a web interface is hosted directly on the Arduino Nano ESP32 that allows for any device to connect to the WiFi access point through a browser. Much of the code is for formatting the web page itself and for visuals. However, additionally, the web page takes user input, so it sends a request back to the microcontroller, which processes the commands and performs the necessary actions. The flash memory section is used to store the users' saved cocktails using JSON strings, which are a sequence of characters representing structured data. This structure is primarily used for data interchange between a server and the web. Ideal for this application. This feature allows the system, on startup, to reload all previously saved cocktails.

```
1 // Define Motor Driver Pins as Outputs
2 void setupPins() {
3     for(int i=0;i<12;i++){
4         pinMode(inPins[i],OUTPUT);
5         digitalWrite(inPins[i],LOW);
6     }
7 }
8
9 void setup() {
10     // Initialize Serial monitor and Pins
11     Serial.begin(115200);
12     delay(1000);
13     setupPins();
14
15     //Initialize LCD and Perform Startup
16     lcd.init();
17     lcd.backlight();
18     lcd.clear();
19     lcd.setCursor(0,0);
20     lcd.print("Cocktail_Machine");
21     lcd.setCursor(0,1);
22     lcd.print("Starting_AP...");
23     delay(3000);
24
25     // Enable WI-FI AP
26     const char* ap_ssid = "CocktailMaker";
27     const char* ap_pass = "mixdrink123";
28     WiFi.mode(WIFI_AP);
29     WiFi.softAP(ap_ssid,ap_pass);
30     Serial.println("AP_started!");
31     Serial.print("IP:"); Serial.println(WiFi.softAPIP());
32
33     // Display AP To User
34     lcd.clear();
```



```

35  lcd.setCursor(0,0); lcd.print("AP_Enabled");
36  lcd.setCursor(0,1); lcd.print("SSID:"); lcd.print(ap_ssid);
37  lcd.setCursor(0,2); lcd.print("PASS:"); lcd.print(ap_pass);
38  lcd.setCursor(0,3); lcd.print("IP:"); lcd.print(WiFi.softAPIP().
    toString());
39
40  // Initialize Time
41  startTime = millis();
42
43  //Load User Stored Cocktails
44  loadCocktailsFromFlash();
45
46  // Start Web Endpoints and Server
47  server.on("/", handleRoot);
48  server.on("/status", handleStatus);
49  server.on("/getCocktails", handleGetCocktails);
50  server.on("/saveCocktail", HTTP_POST, handleSaveCocktail);
51  server.on("/dispense", handleDispense);
52  server.on("/dispenseCocktail", handleDispenseCocktail);
53  server.on("/deleteCocktail", HTTP_POST, handleDeleteCocktail);
54  server.on("/cleanse", handleCleanse);
55  server.begin();
56  Serial.println("Web_server_started");
57 }

```

The setup function consists of initializing the serial monitor and calling the function setupPins, which simply loops through all input pins of the motor drivers in an array and defines them as system outputs, writing them LOW. Next communication with the LCD is started, and the start-up screen is displayed, indicating the system is starting up. The ESP32 WiFi AP is then enabled and displayed on the LCD. A variable is defined that stores the current start time and loads all stored cocktails from the flash memory. Next, the web server routes are created to allow the web interface to interact with the microcontroller.

```

1  void loop() {
2    // Handle Request from Web UI
3    server.handleClient();
4
5    // Store Current Time
6    unsigned long now=millis();
7
8    // Check if Pumps Need to Stop
9    for(int i=0;i<6;i++){
10       if(pumpRunning[i] && now>=pumpEndTime[i]){
11          stopPump(i);
12       }
13    }
14 }

```

The void loop is constantly checking for incoming requests from the browser. When an action from the web page occurs, the microcontroller reads the request and triggers the appropriate server route. Additionally, the code stores the current time. Finally, the loop function checks to see if each pump should be off using a for loop. This helps ensure the pumps are off when expected.

7 Discussion

Overall, the Pour Pal 2.0 successfully met its design goals, with most major design decisions proving effective in practice. The ESP32 microcontroller paired with the web-based interfacing provided intuitive control over the dispensing system. The laser-cut packaging represented a significant improvement over past prototypes. Furthermore, the addition of four new peristaltic pumps with improved flow rates enables quicker dispensing and greater variation in drink selection.

Despite these achievements, several limitations emerged during assembly, testing, and operation. Although most of the wood panels fit together well, there were a few initial errors, such as the wrong sizing for the LCD, and a few panels did not have precise panel alignment, which resulted in needing to remake some parts. While the exit point for the hoses to insert into the drinks is ideal, the length of the hoses themselves is on the shorter side, which may pose challenges when attempting to draw liquid from specific bottles or containers.

Several design tradeoffs became apparent during this iteration. The use of 3mm plywood was cheap and lightweight but had a limited structural rigidity and moisture resistance compared to alternatives such as acrylic. The limited time to allocate a funnel forced the team to get creative with the dispensing system. The hoses are tied together and packaged within the black ring. This current solution causes liquids to dispense awkwardly rather than be a continuous stream.

Future iterations would benefit from several key improvements. A better cable management solution would simplify assembly, maintenance, and troubleshooting. Easing the accessibility of the internals to the user. Upgrading moisture protection both internally and externally improves the durability of the packaging and reliability of the circuitry to prevent deterioration of the device.

8 Work Distribution

The team worked tirelessly on this project in a uniform and effective manner. By efficiently communicating with one another, the project was divided into subsections to be worked on in parallel. Each member took responsibility for a subsection, ensuring the proper work got done and receiving help if needed. Daniel took the responsibility of implementing the code and web interface of the microcontroller, Adam worked on the circuitry of the machine, and Carter designed the packaging. The team came together to finalize the product construction and test the final iterations.

Reporting the process also got divided up between the team members, in which Carter provided the introduction, system overview, packaging description, and conclusion. Adam took responsibility for explaining the hardware and circuit diagram, also discussing the operation of the code, and the distribution of work among the team. Additionally, Carter and Adam collaborated on the discussion. Daniel explained the visuals and operation of the web interface. The team reviewed each other's work constantly and all collaborated to finalize the documents. The link attached directs you to the GitHub Repository: <https://github.com/dnalepa04/Pour-Pal>.

9 Conclusion

The Pour Pal 2.0 project successfully brought together the electrical, mechanical, and software components needed to create a functional automated drink-dispensing system. The ESP32, motor drivers, and pump setup worked reliably, and the web interface made the machine easy to control without needing any external hardware. Just as important, the laser-cut enclosure gave this version a much more polished and intentional feel compared to earlier prototypes. Features like the removable roof, magnetic back door, and dedicated cutouts for tubing and power made the system easier to access, maintain, and operate.

While Pour Pal 2.0 meets all of its main goals, there is still plenty of room for future improvements. More precise flow control, cleaner cable routing, and a more refined user interface would all help push

the design further. Even so, this version represents a solid step forward, showing how a simple idea can be turned into a well-packaged, reliable device with a clear path for continued development.

References

- [1] dnalepa04, “Pour-pal: Automated drink dispenser project.” <https://github.com/dnalepa04/Pour-Pal>, 2025. Accessed: December 10, 2025.

10 Appendix

Computer-Aided Generated Flash Memory Storage and HTML Web Sever/UI Code

```
1 // Save User Cocktails into Flash Memory using JSON Strings
2 void saveCocktailsToFlash() {
3     prefs.begin("cocktails", false);
4     prefs.putInt("count", cocktailCount);
5     for(int i=0; i<cocktailCount; i++){
6         DynamicJsonDocument doc(512);
7         doc["name"] = storedCocktails[i].name;
8         JsonArray vols = doc.createNestedArray("volumes");
9         JsonArray names = doc.createNestedArray("names");
10        for(int j=0; j<6; j++){
11            vols.add(storedCocktails[i].volumes[j]);
12            names.add(storedCocktails[i].pumpNames[j]);
13        }
14        String jsonStr;
15        serializeJson(doc, jsonStr);
16        prefs.putString(("cocktail"+String(i)).c_str(), jsonStr);
17    }
18    prefs.end();
19 }
20
21 // Save User Cocktails From Flash Memory using JSON Strings
22 void loadCocktailsFromFlash() {
23     prefs.begin("cocktails", true);
24     cocktailCount = prefs.getInt("count", 0);
25     for(int i=0; i<cocktailCount; i++){
26         String jsonStr = prefs.getString(("cocktail"+String(i)).c_str(), "");
27         if(jsonStr.length()>0){
28             DynamicJsonDocument doc(512);
29             deserializeJson(doc, jsonStr);
30             storedCocktails[i].name = doc["name"].as<String>();
31             JsonArray vols = doc["volumes"];
32             JsonArray names = doc["names"];
33             for(int j=0; j<6; j++){
34                 storedCocktails[i].volumes[j] = vols[j];
35                 storedCocktails[i].pumpNames[j] = names[j].as<String>();
36             }
37         }
38     }
39     prefs.end();
40 }
41
42 // HTML Page for Cocktail Dispenser UI
43 void handleRoot() {
44     server.send(200, "text/html", R"rawliteral(
45 <!DOCTYPE_html>
46 <html_lang="en">
47 <head>
48 <meta_charset="UTF-8">
```

```

49 <title>ESP32_Cocktail_Dispenser</title>
50 <style>
51 body{font-family:_Arial;_margin:20px;_background:#f4f4f9;_}
52 h1{color:#333;_}
53 section{background:#fff;_padding:15px;_margin-bottom:20px;_border-
    radius:10px;_box-shadow:0_0_5px_#aaa;_}
54 label{display:block;_margin-top:10px;_}
55 input[type=range]{width:100%;_}
56 button{margin-top:10px;_padding:10px_20px;_border:none;_background:#4
    CAF50;_color:white;_border-radius:5px;_cursor:pointer;_}
57 button:hover{background:#45a049;_}
58 .redBtn{background:red;_color:white;_}
59 .redBtn:hover{background:#cc0000;_}
60 select{width:100%;_padding:5px;_margin-top:5px;_}
61 #status_span{font-weight:bold;_}
62 </style>
63 </head>
64 <body>
65 <h1>ESP32_Cocktail_Dispenser</h1>
66
67 <section>
68 <h2>Manual_Control</h2>
69 <form_id="manualForm">
70 <label>Pump_1:<span_id="vol1">0</span>_mL</label>
71 <input_type="range"_min="0"_max="100"_value="0"_id="pump1">
72 <label>Pump_2:<span_id="vol2">0</span>_mL</label>
73 <input_type="range"_min="0"_max="100"_value="0"_id="pump2">
74 <label>Pump_3:<span_id="vol3">0</span>_mL</label>
75 <input_type="range"_min="0"_max="100"_value="0"_id="pump3">
76 <label>Pump_4:<span_id="vol4">0</span>_mL</label>
77 <input_type="range"_min="0"_max="100"_value="0"_id="pump4">
78 <label>Pump_5:<span_id="vol5">0</span>_mL</label>
79 <input_type="range"_min="0"_max="100"_value="0"_id="pump5">
80 <label>Pump_6:<span_id="vol6">0</span>_mL</label>
81 <input_type="range"_min="0"_max="100"_value="0"_id="pump6">
82 <button_type="button"_onclick="dispenseManual()">Dispense</button>
83 <button_type="button"_class="redBtn"_onclick="cleanseTubing()">Cleanse_
    Tubing</button>
84 </form>
85 </section>
86
87 <section>
88 <h2>Create_Custom_Cocktail</h2>
89 <form_id="saveForm">
90 <label>Cocktail_Name:</label>
91 <input_type="text"_id="cocktailName"_placeholder="Enter name">
92 <label>Pump_Liquid_Names_(1-6)</label>
93 <input_type="text"_id="n1"_placeholder="Pump 1 name">
94 <input_type="text"_id="n2"_placeholder="Pump 2 name">
95 <input_type="text"_id="n3"_placeholder="Pump 3 name">
96 <input_type="text"_id="n4"_placeholder="Pump 4 name">

```

```

97 <input_type="text" _id="n5" _placeholder="Pump 5 name">
98 <input_type="text" _id="n6" _placeholder="Pump 6 name">
99 <label>Pump_Volumes_(0-100_mL)</label>
100 <input_type="number" _id="c1" _min="0" _max="100" _value="0">
101 <input_type="number" _id="c2" _min="0" _max="100" _value="0">
102 <input_type="number" _id="c3" _min="0" _max="100" _value="0">
103 <input_type="number" _id="c4" _min="0" _max="100" _value="0">
104 <input_type="number" _id="c5" _min="0" _max="100" _value="0">
105 <input_type="number" _id="c6" _min="0" _max="100" _value="0">
106 <button_type="button" _onclick="saveCocktail()">Save_Cocktail</button>
107 </form>
108 </section>
109
110 <section>
111 <h2>Stored_Cocktails</h2>
112 <select_id="cocktailList" _size="5"></select><br>
113 <button_onclick="dispenseStored()">Dispense_Selected</button>
114 <button_onclick="deleteStored()">Delete_Selected</button>
115 </section>
116
117 <section>
118 <h2>About_/_Status</h2>
119 <div_id="status">
120 <p>IP_Address:_<span_id="ip">-</span></p>
121 <p>Uptime:_<span_id="uptime">-</span></p>
122 <p>WiFi:_<span_id="wifi">-</span></p>
123 <p>Pumps:</p>
124 <ul>
125 <li>Pump_1:_<span_id="p1">Idle</span></li>
126 <li>Pump_2:_<span_id="p2">Idle</span></li>
127 <li>Pump_3:_<span_id="p3">Idle</span></li>
128 <li>Pump_4:_<span_id="p4">Idle</span></li>
129 <li>Pump_5:_<span_id="p5">Idle</span></li>
130 <li>Pump_6:_<span_id="p6">Idle</span></li>
131 </ul>
132 </div>
133 </section>
134
135 <script>
136 function_updateSliders()_{
137   _for(let_i=1;i<=6;i++){
138     _let_s=document.getElementById('pump'+i);
139     _let_v=document.getElementById('vol'+i);
140     _v.textContent=s.value;
141     _s.oninput=()=>{v.textContent=s.value;}
142   _}
143 }
144
145 async_function_fetchStatus(){
146   _try{
147     _let_resp=await_fetch('/status');

```

```

148 |     let_data=await_resp.json();
149 |     document.getElementById('uptime').textContent=data.uptime;
150 |     document.getElementById('ip').textContent=data.ip;
151 |     document.getElementById('wifi').textContent=data.wifi;
152 |     for (let_i=0;i<6;i++){
153 |         document.getElementById('p'+(i+1)).textContent=data.pumps[i]?'
        Running':'Idle';
154 |     }
155 | }catch(e){console.log(e);}
156 | }
157 |
158 | async_function_fetchCocktails(){
159 |     try{
160 |         let_resp=await_fetch('/getCocktails');
161 |         let_data=await_resp.json();
162 |         let_list=document.getElementById('cocktailList');
163 |         list.innerHTML='';
164 |         data.forEach(c=>{
165 |             let_opt=document.createElement('option');
166 |             opt.value=c.name;
167 |             opt.text=c.name;
168 |             list.appendChild(opt);
169 |         });
170 |     }catch(e){console.log(e);}
171 | }
172 |
173 | async_function_dispenseManual(){
174 |     let_params=[];
175 |     for (let_i=1;i<=6;i++){
176 |         params.push('m'+i+'='+document.getElementById('pump'+i).value);
177 |     }
178 |     await_fetch('/dispense?'+params.join('&'));
179 |     fetchStatus();
180 | }
181 |
182 | async_function_saveCocktail(){
183 |     let_obj={name:document.getElementById('cocktailName').value,volumes:[],
        names:[]};
184 |     for (let_i=1;i<=6;i++){
185 |         obj.names.push(document.getElementById('n'+i).value);
186 |         obj.volumes.push(Number(document.getElementById('c'+i).value));
187 |     }
188 |     await_fetch('/saveCocktail',{
189 |         method:'POST',
190 |         headers:{'Content-Type':'application/json'},
191 |         body:JSON.stringify(obj)
192 |     });
193 |     fetchCocktails();
194 | }
195 |
196 | async_function_dispenseStored(){

```

```

197 __let_sel=document.getElementById('cocktailList').value;
198 __if(sel)_await_fetch('/dispenseCocktail?name='+encodeURIComponent(sel));
199 __fetchStatus();
200 }
201
202 async_function_deleteStored(){
203 __let_sel=document.getElementById('cocktailList').value;
204 __if(sel)_await_fetch('/deleteCocktail?name='+encodeURIComponent(sel),{
205     method:'POST'});
206 __fetchCocktails();
207 }
208
209 async_function_cleanseTubing(){
210 __await_fetch('/cleanse');
211 __fetchStatus();
212 }
213
214 updateSliders();
215 setInterval(fetchStatus,1000);
216 setInterval(fetchCocktails,5000);
217 fetchStatus();
218 fetchCocktails();
219 </script>
220 </body>
221 </html>
222 )rawliteral");
223 }
224
225 // Return Status of Pour Pal as JSON String
226 void handleStatus() {
227     String wifiStatus="Access_Point";
228     String json="{\"uptime\":\":"+getUptime()+"\", \"ip\":\":"+WiFi.softAPIP()
229         .toString()+"\", \"wifi\":\":"+wifiStatus+"\", \"pumps\":[";
230     for(int i=0;i<6;i++){
231         json+=pumpRunning[i]?\"true\":\"false\";
232         if(i<5) json+=\",\";
233     }
234     json+=\"]}\";
235     server.send(200,\"application/json\",json);
236 }
237
238 // Return Stored Cotaills as JSON Array to Populate UI
239 void handleGetCocktails(){
240     String json="[";
241     for(int i=0;i<cocktailCount;i++){
242         json+=\"{\\\"name\\\":\":"+storedCocktails[i].name+"\", \"volumes\":[";
243         for(int j=0;j<6;j++){
244             json+=String(storedCocktails[i].volumes[j]);
245             if(j<5) json+=\",\";
246         }
247         json+=\"],\\\"names\\\":\":";

```



```

246     for(int j=0;j<6;j++){
247         json+="\""+storedCocktails[i].pumpNames[j]+"\"";
248         if(j<5) json+=",";
249     }
250     json+="]";
251     if(i<cocktailCount-1) json+=",";
252 }
253 json+="]";
254 server.send(200,"application/json",json);
255 }
256
257 // Saves New User Cocktails
258 void handleSaveCocktail(){
259     if(cocktailCount>=MAX_COCKTAILS){server.send(200,"text/plain","Max_
        cocktails_reached");return;}
260     if(!server.hasArg("plain")){server.send(400);return;}
261     String body=server.arg("plain");
262     DynamicJsonDocument doc(1024);
263     deserializeJson(doc,body);
264     String name=doc["name"];
265     JSONArray vols=doc["volumes"];
266     JSONArray names=doc["names"];
267     storedCocktails[cocktailCount].name=name;
268     for(int i=0;i<6;i++){
269         storedCocktails[cocktailCount].volumes[i]=vols[i];
270         storedCocktails[cocktailCount].pumpNames[i]=names[i].as<String>();
271     }
272     cocktailCount++;
273     saveCocktailsToFlash();
274     server.send(200,"text/plain","OK");
275 }
276
277 // Dispense Manual Volumes from User
278 void handleDispense(){
279     for(int i=0;i<6;i++){
280         if(server.hasArg("m"+String(i+1))){
281             int v=server.arg("m"+String(i+1)).toInt();
282             if(v>0) startPump(i,v); else stopPump(i);
283         }
284     }
285     server.send(200,"text/plain","OK");
286 }
287
288 // Dispense Saved User Cocktail
289 void handleDispenseCocktail(){
290     if(!server.hasArg("name")){server.send(400);return;}
291     String name=server.arg("name");
292     for(int i=0;i<cocktailCount;i++){
293         if(storedCocktails[i].name==name){
294             for(int j=0;j<6;j++){
295                 int vol=storedCocktails[i].volumes[j];

```

```

296         if(vol>0) startPump(j,vol);
297     }
298     server.send(200,"text/plain","OK");return;
299 }
300 }
301 server.send(404,"text/plain","Cocktail_not_found");
302 }
303
304 // Delete Saved User Cocktail
305 void handleDeleteCocktail(){
306     if(!server.hasArg("name")){server.send(400);return;}
307     String name=server.arg("name");
308     for(int i=0;i<cocktailCount;i++){
309         if(storedCocktails[i].name==name){
310             for(int j=i;j<cocktailCount-1;j++) storedCocktails[j]=
                storedCocktails[j+1];
311             cocktailCount--;
312             saveCocktailsToFlash();
313             server.send(200,"text/plain","Deleted");return;
314         }
315     }
316     server.send(404,"text/plain","Not_found");
317 }
318
319 // Run All Pumps in Reverse for 5 Seconds to Clean Tubes
320 void handleCleanse(){
321     unsigned long duration=5000;
322     unsigned long endTime=millis()+duration;
323     for(int i=0;i<6;i++){
324         hardwareReversePump(i);
325         pumpRunning[i]=true;
326         pumpEndTime[i]=endTime;
327     }
328     server.send(200,"text/plain","Cleaning_in_progress");
329 }

```